

ECE 4974 Lab - Dynamic Sanitization of json-c with ASan + UBSan

Los Escalante Target: json-c (github.com/json-c/json-c)

1. Target choice and justification

json-c is a mature, widely deployed C library for parsing and building JSON. It is a good Lab 1 target since it is real production code, it builds cleanly with CMake and Clang, and it has been fuzzed constantly by Google's OSS-Fuzz since 2017. That last point shaped the whole investigation, since the heavily-fuzzed parser is hardened, so the defects that remain live on the library's API surface. That contrast is the central finding of this lab.

2. Build process

Two builds were produced per versio. The first is a normal release build to confirm the library works on valid input, and an instrumented Debug build with the Part 2 flag set. Findings F2 and F3 required an older release (0.13.1), built with the same flags in a separate tree. Toolchain: Clang/LLVM 21.1.8 on Kali Linux.

The flag set is exactly as specified on slide 6. The library was built as a static archive (`BUILD_SHARED_LIBS=OFF`) so the sanitizer runtime is linked into each test binary, which is what makes the `nm` evidence below significant.

```
cmake -DCMAKE_POLICY_VERSION_MINIMUM=3.5 \  
  -DCMAKE_BUILD_TYPE=Debug -DCMAKE_C_COMPILER=clang \  
  -DBUILD_SHARED_LIBS=OFF -DDISABLE_WERROR=ON \  
  -DCMAKE_C_FLAGS="-fsanitize=address,undefined -fno-omit-frame-pointer \  
    -fno-optimize-sibling-calls -fsanitize-address-use-after-scope -g -O1" \  
  -DCMAKE_EXE_LINKER_FLAGS="-fsanitize=address,undefined" ..  
make -j$(nproc) 2>&1 | tee build.log  
  
clang -fsanitize=address,undefined -fno-omit-frame-pointer -g -O1 \  
  -Ibuild-asan -Ibuild-asan/include -I. trigger.c build-asan/libjson-c.a -o trigger
```

Two build system traits were specific to these releases: `-DCMAKE_POLICY_VERSION_MINIMUM=3.5` is required because modern CMake rejects the old `cmake_minimum_required` version json-c declares; and json-c 0.13.x appends `-Werror` after the user's flags, so `DISABLE_WERROR` alone is insufficient. The specific Clang 21 warnings had to be silenced (`-Wno-implicit-const-int-float-conversion` and two related flags) so the appended `-Werror` has nothing to promote.

Deliverable B - Build artifacts

Instrumentation is proven in the binary. The slide 27 pitfall is that `ldd` proves nothing here since Clang links the runtime statically, the correct check is `nm`.

```
$ nm -C ./trigger | grep -c __asan  
698  
$ ldd ./trigger | grep -c asan  
0
```

The 698 `__asan` symbols confirm instrumentation, the empty `ldd` result is expected and is itself the evidence that a naive `ldd` check would mislead. The build command, `make log` (`build.log`), and this `nm` output are committed in each build tree.

3. Methodology

The instrumented parser was driven with 3 input sources per the task: the project's own regression corpus, hand-made edge cases, and reference to a public CVE (Section 6). A small script harvests every JSON test string from json-c's own `test_parse.c`, copies the two shipped fixtures, and adds 11 edge cases I craft of 3000-level nested arrays, a 39-digit integer, `1e9999999999`, truncated input, an embedded NUL, 2000 duplicate keys, a lone `\u` escape, a 200,000-character string, and deeply nested empty keys.

All 150 inputs ran through the tokenizer under the development ASan/UBSan configuration. Every one parsed or rejected cleanly, zero sanitizer reports. This is not a null result, as it matches json-c's continuous OSS-Fuzz coverage and redirected the investigation to the API surface, where the fuzzer doesn't supply hostile arguments. All three findings were reached that way.

4. Findings

Three distinct defects were found. Distinct bug classes, distinct source sites, each fixed by a distinct upstream commit. F1 is present in current 0.15; F2 and F3 are in 0.13.1, fixed in later releases. The three are unique, none are duplicate.

#	Version	Class	Site	Upstream fix
F1	0.15	allocation-size-too-big	array_list_new2 via json_object_new_array_ext	369e847
F2	0.13.1	SEGV via size_t overflow	array_list_del_idx → array_list_free	099016b
F3	0.13.1	calloc-overflow	json_tokenizer_new_ex	later depth validation

5. Findings details and root-cause analysis

F1 allocation-size-too-big in array_list_new2 (0.15)

Trigger: json_object_new_array_ext(INT_MIN + 1). Reproduced by trigger.c in the 0.15 tree.

```
==30479==ERROR: AddressSanitizer: requested allocation size 0xffffffff00000008 (0xffffffff00001008 after adjustments
→ for alignment, red zones etc.) exceeds maximum supported size of 0x10000000000 (thread T0)
#0 0x562d35023a58 in malloc (/home/loresc/ece4974/lab1/json-c/trigger+0x102a58) (BuildId:
→ 1c3774b421e48e402cee0e77d0f46a922514ba02)
#1 0x562d35080ba0 in array_list_new2 /home/loresc/ece4974/lab1/json-c/arraylist.c:54:30
#2 0x562d3506fc25 in json_object_new_array_ext /home/loresc/ece4974/lab1/json-c/json_object.c:1441:17
#3 0x562d35067c15 in main /home/loresc/ece4974/lab1/json-c/trigger.c:4:29
#4 0x7f46264bdf76 (/usr/lib/x86_64-linux-gnu/libc.so.6+0x29f76) (BuildId:
→ fdce52df7ab391982f6de31414f29957668212ff)

==30479==HINT: if you don't care about these errors you may set allocator_may_return_null=1
SUMMARY: AddressSanitizer: allocation-size-too-big (/home/loresc/ece4974/lab1/json-c/trigger+0x102a58) (BuildId:
→ 1c3774b421e48e402cee0e77d0f46a922514ba02) in malloc
==30479==ABORTING
```

Root cause - The public API json_object_new_array_ext accepts the initial size as a signed int and passes it unvalidated to array_list_new2, which computes malloc(size * sizeof(void *)). A negative argument converts to a very large size_t in that multiplication. On this system the request exceeds ASan's maximum allocation and is caught as allocation-size-too-big. The deeper danger is platform-dependent. On 32-bit the multiplication would wrap to a small value, malloc would succeed with an undersized buffer, and every following element write would be an out-of-bounds heap write. Fix 369e847 ("Validate size arguments in arraylist functions") adds an explicit bound rejecting negative and overflowing sizes before allocating.

F2 - size_t overflow in array_list_del_idx (0.13.1)

Trigger: array_list_del_idx(a, 1, (size_t)-1) on a one-element list. Reproduced by poc_F2_del_idx.c.

```
AddressSanitizer:DEADLYSIGNAL
=====
==30457==ERROR: AddressSanitizer: SEGV on unknown address 0xffffffffffffff8 (pc 0x555b05f57f19 bp 0x7ffde1021ea0 sp
→ 0x7ffde1021e40 T0)
==30457==The signal is caused by a READ memory access.
#0 0x555b05f57f19 in array_list_free /home/loresc/ece4974/lab1/json-c-0131/arraylist.c:61:23
#1 0x555b05f57b00 in main /home/loresc/ece4974/lab1/json-c-0131/poc_F2_del_idx.c:9:5
#2 0x7ff2cc2a0f76 (/usr/lib/x86_64-linux-gnu/libc.so.6+0x29f76) (BuildId:
→ fdce52df7ab391982f6de31414f29957668212ff)
#3 0x7ff2cc2a1026 in __libc_start_main (/usr/lib/x86_64-linux-gnu/libc.so.6+0x2a026) (BuildId:
→ fdce52df7ab391982f6de31414f29957668212ff)
#4 0x555b05e6e360 in _start (/home/loresc/ece4974/lab1/json-c-0131/poc_F2+0x2e360) (BuildId:
→ a9b6f90331a15e3b5c46d94b7973d88dd9236d60)

==30457==Register values:
rax = 0x00000f84596bc00b  rbx = 0x10000000000000000  rcx = 0x0000000000000000  rdx = 0x0000000000000000
```

```
rdi = 0x00007c22cb5e0040  rsi = 0x00007c22cb5e0058  rbp = 0x00007ffde1021ea0  rsp = 0x00007ffde1021e40
r8 = 0x00007c22cb5e0048  r9 = 0x0000000000000000  r10 = 0x00007fffffff01  r11 = 0xffffffff00000fff01
r12 = 0x00000f84596bc008  r13 = 0x00007d02cb5e0040  r14 = 0x0000000000000000  r15 = 0x00007d02cb5e0040
AddressSanitizer can not provide additional info.
SUMMARY: AddressSanitizer: SEGV /home/lolesec/ece4974/lab1/json-c-0131/arraylist.c:61:23 in array_list_free
==30457==ABORTING
```

Relevant source (arraylist.c:134):

```
stop = idx + count;
if (idx >= arr->length || stop > arr->length) return -1;
...
arr->length -= count;
```

Root cause -The guard computes `stop = idx + count` then rejects if `stop > arr->length`. With `idx = 1` and `count = SIZE_MAX`, the addition wraps to `stop = 0`, so `0 > 1` is false and the check is defeated, thus the integer-overflow-bypasses-bounds-check pattern. Execution reaches `arr->length -= count`, which underflows the length to a near-SIZE_MAX value, leaving the array corrupt. The crash occurs later, when `array_list_free` walks the array using that length and reads off the end into unmapped memory, the SEGV at `arraylist.c:61` on address `0xffffffffffffff8`. Fix 099016b (“Protect array_list_del_idx against size_t overflow”) adds an overflow guard before the subtraction.

F3 - alloc overflow from negative depth in json_tokenizer_new_ex (0.13.1)

Trigger: `json_tokenizer_new_ex(-1)`. Reproduced by `poc_F3_neg_depth.c`.

```
=====
==30459==ERROR: AddressSanitizer: calloc parameters overflow: count * size (-1 * 32) cannot be represented in type
↳ size_t (thread T0)
#0 0x55df62e6dcdb in calloc (/home/lolesec/ece4974/lab1/json-c-0131/poc_F3+0x100cbd) (BuildId:
↳ 9941d6ebb4ff5af184595d631c556e5ae2dcd9)
#1 0x55df62ebd373 in json_tokenizer_new_ex /home/lolesec/ece4974/lab1/json-c-0131/json_tokenizer.c:115:45
#2 0x55df62eb1ca8 in main /home/lolesec/ece4974/lab1/json-c-0131/poc_F3_neg_depth.c:4:30
#3 0x7f476eeaff76 (/usr/lib/x86_64-linux-gnu/libc.so.6+0x29f76) (BuildId:
↳ fdce52df7ab391982f6de31414f29957668212ff)

==30459==HINT: if you don't care about these errors you may set allocator_may_return_null=1
SUMMARY: AddressSanitizer: calloc-overflow (/home/lolesec/ece4974/lab1/json-c-0131/poc_F3+0x100cbd) (BuildId:
↳ 9941d6ebb4ff5af184595d631c556e5ae2dcd9) in calloc
==30459==ABORTING
```

Root cause - The tokenizer constructor takes a maximum nesting `depth` as a signed `int` and passes it directly to `calloc(depth, sizeof(struct json_tokenizer_srec))` with no positivity check. A negative depth converts to a large `size_t` count; ASan’s allocator detects that `count * size` can’t be represented and reports `calloc-overflow` before returning memory. Even if satisfiable, a senseless depth would corrupt the tokenizer’s stack bookkeeping, since `depth` also bounds later `tok->stack[...]` indexing. Later releases validate `depth` on entry.

Deliverable A - ASAN_OPTIONS / UBSAN_OPTIONS

The development recipe from slide 15 was used for all runs, since the goal was to reproduce and inspect each bug.

```
ASAN_OPTIONS="detect_leaks=0:print_stacktrace=1"
UBSAN_OPTIONS="print_stacktrace=1:halt_on_error=1"
```

- `detect_leaks=0` - suppresses LeakSanitizer, whose cleanup reports are noise during single-shot reproduction.
- `print_stacktrace=1` (both) - ASan defaults this on but UBSan does not, without it a UBSan finding is a single line, pretty much useless for triage. Set on both for consistency.
- `halt_on_error=1` (UBSan) - makes UBSan stop on first error like ASan rather than printing and continuing.

The F1 and F3 reports print HINT: `... allocator_may_return_null=1`. That option was left at its default (0). Setting it would make the huge allocations return NULL silently and hide exactly the defects being shown, so leaving it off is fine for this workflow.

6. References

- AddressSanitizer - <https://clang.llvm.org/docs/AddressSanitizer.html>

- UndefinedBehaviorSanitizer - <https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html>
- ASAN_OPTIONS flags - <https://github.com/google/sanitizers/wiki/AddressSanitizerFlags>
- json-c repository - <https://github.com/json-c/json-c>
- Fix 369e847 (F1) - <https://github.com/json-c/json-c/commit/369e847>
- Fix 099016b (F2) - <https://github.com/json-c/json-c/commit/099016b>
- CVE-2020-12762 - <https://nvd.nist.gov/vuln/detail/CVE-2020-12762>

Note on CVE PoC input: CVE-2020-12762 is present in json-c 0.13.1, but triggering it requires growing a buffer past INT_MAX , on the order of 2 GB of data, which is not practical on my laptop. It is cited as the public CVE reference rather than something that was reproduced. The three findings above are the reproduced defects.